Type-I-Precoding und Beamforming in OCUDU

Erklärungen mit Quellcode-Ausschnitten

Zusammengestellt aus einer Code-Analyse des Repositories

6. November 2026

Inhaltsverzeichnis

1 Wo die Type-I-Codebooks erzeugt werden **1**
- 1.1 Die Mathematik: eine Matrix aus einem PMI . . . 1
- 1.2 Das komplette Codebook für die Zelle . . . 2
- 1.3 Konfiguration und Nutzung im Scheduler . . . 3

2 OCUDU mit PDSCH-Traffic starten und die Generierung beobachten **3**
- 2.1 Bauen . . . 3
- 2.2 Konfiguration: Testmodus mit 4 Ports . . . 3
- 2.3 Breakpoints für die Generierung . . . 5
- 2.4 Die Nutzung im laufenden Traffic . . . 5

3 3GPP-Codebook und Beam-Codebook: zwei verschiedene W **5**
- 3.1 Das 3GPP-Codebook . . . 5
- 3.2 Das Beam-Codebook . . . 5
- 3.3 Die Zerlegung . . . 7
- 3.4 Wo die Multiplikation mit den IQ-Samples passiert . . . 7
- 3.5 Wann `generate_beam_weights_codebook()` aufgerufen wird . . . 8
- 3.6 Wofür das Beam-Codebook trotzdem existiert . . . 8
- 3.7 Der UHD-SDR-Fall . . . 9
- 3.8 Was OCUDU beim Beamforming nicht macht . . . 9

4 Die Mathematik von `make_type1_sp_mode1()` **9**
- 4.1 Antennenmodell . . . 9
- 4.2 Der Steuervektor $v_{l,m}$ . . . 10
- 4.3 Die Co-Phasierung φ_n . . . 11
- 4.4 Ein Layer . . . 11
- 4.5 Zwei Layer . . . 11
- 4.6 Drei und vier Layer . . . 11
- 4.7 Normierung . . . 12
- 4.8 Zahlenbeispiel . . . 12
- 4.9 Was das Precoding damit macht . . . 12

5 Die beiden Dateien `precoding_codebooks.cpp` **13**
- 5.1 `lib/ran/precoding/precoding_codebooks.cpp`, die Fassade . . . 13
- 5.2 `lib/ran/precoding/type1_sp/precoding_codebooks.cpp`, die Umsetzung . . . 13
- 5.3 Warum die Aufteilung . . . 14

6 Die Tests: Parametrisierung und Matrix-Ausgabe **14**
6.1 Wer über die Topologien iteriert 14
6.2 Beam-Gewichte als Matrix ausgeben 15

Alle Pfade sind relativ zur Wurzel des Repositories. Zeilennummern beziehen sich auf den Stand vom 6. November 2026 (Commit `9f591a95fc`) und können sich mit späteren Änderungen verschieben.

1 Wo die Type-I-Codebooks erzeugt werden

Die Type-I-Codebooks sind in zwei Schichten organisiert. Die Matrix-Mathematik liegt in `lib/ran/precoding/`, die Erzeugung des kompletten Codebook-Repositories für eine Zelle im FAPI-Adaptor.

1.1 Die Mathematik: eine Matrix aus einem PMI

- `lib/ran/precoding/type1_sp/precoding_codebooks.cpp` ist der Kern. `make_type1_sp_mode1()` baut die Gewichtsmatrix nach TS 38.214 Tabellen 5.2.2.2.1-5 bis -8 für 1 bis 8 Layer. Pro Layer wird `make_layer_type1_sp_mode1()` aufgerufen: Kronecker-Produkt der horizontalen und vertikalen DFT-Vektoren, die zweite Polarisation erhält den Co-Phasing-Faktor φ. `get_k1_and_k2_2_layers()` und `get_k1_and_k2_3_4_layers()` sind die Tabellen 5.2.2.2.1-3 und -4 (Beam-Offset zwischen Layern über $i_{1,3}$).
- Dieselbe Datei enthält `calculate_mimo_matrix()` und `get_beams_from_pmi()`, die zerlegte Variante für Topologien mit Beam-Raster (Abschnitt 3).
- `lib/ran/precoding/two_port/precoding_codebooks.cpp`: Sonderfall 2 Ports, Tabelle 5.2.2.2.1-1, hart codiert.
- `lib/ran/precoding/precoding_codebooks.cpp`: Dispatcher. `make_precoding()` und `get_mimo_matrix` machen ein `std::visit` über die PMI-Variante (Abschnitt 5).
- `lib/ran/precoding/type1_sp/precoding_codebook_helpers.cpp`: Wertebereiche und Bitbreiten der PMI-Felder $i_{1,1}, i_{1,2}, i_{1,3}, i_2$ je Panel und Rang, TS 38.212 Tabelle 6.3.1.1.2-1.
- `lib/ran/precoding/precoding_codebook_configuration.cpp`: `get_single_panel_info()`, die Tabelle N_1, N_2, O_1, O_2 der unterstützten Panels 2×1, 2×2, 4×1.

Die Typen stehen in `include/ocudu/ran/precoding/`: `precoding_matrix_indicator.h` mit `pmi_typeI_single_panel`, `precoding_codebook_configuration.h` und `precoding_codebook_type1_hel`

include/ocudu/ran/precoding/precoding_matrix_indicator.h

```
struct pmi_typeI_single_panel {
  /// Single-panel topology configuration.
  pmi_codebook_typeI_single_panel panel_config;
  /// PMI parameter i_{1,1}. Values: {0,...,N_1*O_1 - 1}, at most {0,...,63}.
  uint8_t i_1_1;
  /// PMI parameter i_{1,2}. Only available for N_2 > 1 or v > 2.
  std::optional<uint8_t> i_1_2;
  /// PMI parameter i_{1,3}. Only available for v in {2,3,4}.
  std::optional<uint8_t> i_1_3;
  /// PMI parameter i_2.
  uint8_t i_2;
};

using precoding_matrix_indicator =
    std::variant<std::monostate, pmi_two_antenna_port, pmi_typeI_single_panel,
    pmi_typeII>;
```

1.2 Das komplette Codebook für die Zelle

`lib/fapi_adaptor/precoding_codebook_generator.cpp`, Funktion `generate_precoding_codebooks()`. Beim Start wird für die Zelle einmal jede PMI-Kombination durchgezählt. In `generate_pdsch_sp_type1()` laufen vier verschachtelte Schleifen über $i_{1,1}, i_{1,2}, i_{1,3}, i_2$, pro Layer-Anzahl 1 bis `nof_ports`, und jede Matrix landet unter einem Precoding-Matrix-Index im Repository.

lib/fapi_adaptor/precoding_codebook_generator.cpp, generate_pdsch_sp_type1()

```
pmi_typeI_single_panel_param_ranges param_ranges = get_pmi_ranges_typeI_single_panel(
    panel, nof_layers);

for (uint8_t i_1_1 = 0; i_1_1 != param_ranges.i_1_1; ++i_1_1) {
  for (uint8_t i_1_2 = 0; i_1_2 != param_ranges.i_1_2; ++i_1_2) {
    for (uint8_t i_1_3 = 0; i_1_3 != param_ranges.i_1_3; ++i_1_3) {
      for (uint8_t i_2 = 0; i_2 != param_ranges.i_2; ++i_2) {
        pmi_typeI_single_panel pmi = {.panel_config = panel,
                                      .i_1_1        = i_1_1,
                                      .i_1_2        = (param_ranges.i_1_2 > 0) ? std
    ::optional(i_1_2) : std::nullopt,
                                      .i_1_3        = (param_ranges.i_1_3 > 0) ? std
    ::optional(i_1_3) : std::nullopt,
                                      .i_2          = i_2};

        unsigned pm_index = base_offset +
    get_pdsch_single_panel_type1_precoding_matrix_index(param_ranges, pmi);
        repo_builder.add(pm_index, make_composite(pmi, nof_layers, topology));
      }
    }
  }
}
```

Daneben werden Identität, SSB, PDCCH, omnidirektionales PDSCH, CSI-RS und PRS eingetragen. `make_composite()` entscheidet anhand der Antennentopologie, ob die volle Port-Matrix oder die Beam-Zerlegung gespeichert wird:

lib/fapi_adaptor/precoding_codebook_generator.cpp, make_composite()

```
static precoding_beamforming_composite
make_composite(const precoding_matrix_indicator& pmi, unsigned nof_layers,
    antenna_topology topology)
{
  if (has_beam_grid(topology)) {
    return get_mimo_matrix_from_pmi(pmi, nof_layers);
  }

  precoding_weight_matrix precoding = make_precoding(pmi, nof_layers);

  return {precoding, get_default_beam_list(precoding.get_nof_ports())};
}
```

Aufgerufen wird die Generierung in `lib/du/du_high/o_du_high_factory.cpp:105` (MAC-Seite, Mapper) und `lib/du/du_low/o_du_low_factory.cpp:64` (PHY-Seite, Repository). Welches Codebook gilt, hängt nur von der Anzahl der Sendeantennen ab:

lib/du/du_low/o_du_low_factory.cpp

```
switch (fapi_cfg.sectors[i].p7_config.carrier_cfg.num_tx_ant) {
  case 1: codebook_config = pmi_codebook_one_port{}; break;
  case 2: codebook_config = pmi_codebook_two_port{}; break;
  case 4:
```

```
    codebook_config =
        pmi_codebook_typeI_single_panel{pmi_codebook_single_panel_config::two_one,
    pmi_codebook_typeI_mode::one};
    break;
  case 8:
    codebook_config =
        pmi_codebook_typeI_single_panel{pmi_codebook_single_panel_config::four_one,
    pmi_codebook_typeI_mode::one};
    break;
}
```

1.3 Konfiguration und Nutzung im Scheduler

- `lib/scheduler/config/csi_helper.cpp`, `make_type1_codebook_config()`: baut die RRC-CodebookConfig für das UE (Subset-Restriction-Bitmaps, Panel 2×1 oder 4×1).
- `lib/scheduler/ue_context/ue_channel_state_manager.cpp`: Default-PMI-Konfigurationen und Speicherung des vom UE gemeldeten PMI.

Zum Einstieg empfiehlt sich die Reihenfolge `make_type1_sp_mode1()` → `generate_pdsch_sp_type1()` → `calculate_mimo_matrix()`.

2 OCUDU mit PDSCH-Traffic starten und die Generierung beobachten

Es gibt zwei Wege: die komplette gNB im Testmodus mit Dummy-RU (kein Core, kein Funk, aber echter PDSCH-Traffic für ein Phantom-UE), oder die Unit-Tests der Codebook-Generierung im Debugger.

2.1 Bauen

```
cmake --build cmake-build-debug --target gnb precoding_codebook_generator_test
    precoding_codebooks_test -j4
```

2.2 Konfiguration: Testmodus mit 4 Ports

Vorlage ist `configs/testmode.yml`. Für Type-I-Codebooks sind mindestens 4 DL-Antennen nötig, sonst wird nur das 2-Port- oder 1-Port-Codebook erzeugt.

configs/testmode_4x4.yml (Beispiel)

```
cu_cp:
  amf:
    no_core: true

ru_dummy:
  dl_processing_delay: 1

cell_cfg:
  dl_arfcn: 632628
  band: 78
  channel_bandwidth_MHz: 20
  common_scs: 30
  plmn: "00101"
  tac: 7
```

```
  pci: 1
  nof_antennas_dl: 4
  nof_antennas_ul: 4
  # Optional: Topologie mit Beam-Raster. Ohne Angabe wird four_ports abgeleitet.
  # tx_ant_topology: single_panel_two_one

test_mode:
  test_ue:
    rnti: 0x44
    nof_ues: 1
    pdsch_active: true
    pusch_active: false
    cqi: 15
    ri: 2          # 1..4, bestimmt die Layer-Anzahl
    i_1_1: 1       # PMI-Felder, die das Phantom-UE "meldet"
    i_1_3: 0
    i_2: 1

log:
  filename: /tmp/gnb.log
  all_level: warning
  mac_level: debug
  phy_level: debug
```

Das Phantom-UE meldet seine CSI aus genau diesen Werten. Das Packen passiert in `lib/du/du_high/test_`

lib/du/du_high/test_mode/mac_test_mode_helpers.cpp, fill_pmi_bits()

```
const auto* single_panel = std::get_if<pmi_codebook_typeI_single_panel>(&pmi_codebook
    );
if (single_panel == nullptr) {
  return;
}

// The fields that the panel topology and the rank do not report have a zero bit-
    width.
const pmi_typeI_single_panel_param_sizes sizes =
    get_pmi_sizes_typeI_single_panel(get_single_panel_info(single_panel->n1_n2), rank
    );
packed.push_back(test_ue_cfg.i_1_1, sizes.i_1_1);
packed.push_back(test_ue_cfg.i_1_2.value_or(0), sizes.i_1_2);
packed.push_back(test_ue_cfg.i_1_3.value_or(0), sizes.i_1_3);
packed.push_back(test_ue_cfg.i_2, sizes.i_2);
```

Starten:

```
./cmake-build-debug/apps/gnb/gnb -c configs/testmode_4x4.yml
```

Die Topologie-Namen liefert `./cmake-build-debug/apps/gnb/gnb cell_cfg -h`. Namen mit Präfix `single_panel_` haben ein Beam-Raster.

2.3 Breakpoints für die Generierung

Die Codebook-Erzeugung loggt nichts und läuft genau einmal beim Start, bevor Traffic fließt.

- `lib/fapi_adaptor/precoding_codebook_generator.cpp:184`, die Zeile `repo_builder.add(pm_index, make_composite(...))` in der vierfachen Schleife.
- `lib/ran/precoding/type1_sp/precoding_codebooks.cpp`, Einstieg von `make_type1_sp_mode1()` und der `switch (nof_layers)`.

- `lib/du/du_low/o_du_low_factory.cpp:64` und `lib/du/du_high/o_du_high_factory.cpp:105`, die beiden Aufrufer.

Isolierter geht es über die Unit-Tests:

```
./cmake-build-debug/tests/unittests/fapi_adaptor/precoding_codebook_generator_test --
    gtest_filter='*TypeI*'
./cmake-build-debug/tests/unittests/ran/precoding_codebooks_test
```

2.4 Die Nutzung im laufenden Traffic

Mit `mac_level: debug` schreibt der Scheduler pro PDSCH-Grant die gewählte PMI ins Log (`lib/scheduler/logging/scheduler_result_logger.cpp:627`, Format `ri=<layers>` gefolgt von den PMI-Feldern). Mit `phy_level: debug` protokolliert der PDSCH-Prozessor die angewendete Precoding-Konfiguration.

```
grep -E "PDSCH.*ri=" /tmp/gnb.log | head
```

Den Weg von der PMI im Scheduler zum Index im Repository zeigt `lib/fapi_adaptor/precoding_codeboo`
Zur Laufzeit wird nichts neu gerechnet: Der PHY-Adaptor holt in `lib/fapi_adaptor/phy/p7/pdu_translator`
mit `get_precoding_config()` das fertige Composite aus dem Repository.

3 3GPP-Codebook und Beam-Codebook: zwei verschiedene W

Es gibt zwei verschiedene „W“ im Code. Die 3GPP-Matrix ist `make_type1_sp_mode1()`. `generate_beam_weigh`
ist *nicht* die Umsetzung des 3GPP-Codebooks.

3.1 Das 3GPP-Codebook

`lib/ran/precoding/type1_sp/precoding_codebooks.cpp`, `make_type1_sp_mode1()`, erzeugt aus dem PMI die Matrix W mit Dimension CSI-RS-Ports $\times$ Layer, genau wie in TS 38.214 Tabelle 5.2.2.2.1-5 bis -8. Pro Layer ein Vektor $[v_{l,m};\ \varphi_n v_{l,m}]$, normiert mit $1/\sqrt{vP}$.

3.2 Das Beam-Codebook

`lib/ran/beamforming/beam_weights_codebook_generator.cpp` steht in keiner 3GPP-Spezifikation. Es ist eine implementierungsspezifische Beam-Tabelle: alle DFT-Richtungen, die das Antennen-Array bilden kann, plus die Einzelport-Beams. Das Konzept stammt aus O-RAN, wo die Radio Unit eine feste Tabelle von Beam-IDs kennt. Zeilen sind Beams, Spalten physische Antennen.

lib/ran/beamforming/beam_weights_codebook_generator.cpp

```
beam_weights_codebook ocudu::generate_beam_weights_codebook(antenna_topology topology
    )
{
  // ... Topologie-Parameter: nof_panels, nof_elements_dim1/2, nof_polarizations,
    nof_beams_dim1/2 ...
  beam_weights_codebook beam_weights = beam_weights_codebook(nof_total_beams,
    nof_total_antennas);

  // Direkte Port-Beams: Einheitsmatrix in den ersten nof_total_antennas Zeilen.
  for (unsigned i_antenna = 0; i_antenna != nof_total_antennas; ++i_antenna) {
    beam_weights.set_coefficient(1.0, get_beam_id(topology, i_antenna), i_antenna);
  }

  // Only direct beam to port mapping if the panels are 1x1 elements.
  if (!has_beam_grid(topology)) {
```

```
    return beam_weights;
  }

  float amplitude = std::sqrt(1.0F / static_cast<float>(nof_elements_dim1 *
    nof_elements_dim2));

  for (unsigned i_panel = 0; i_panel != nof_panels; ++i_panel) {
    for (unsigned i_pol = 0; i_pol != nof_polarizations; ++i_pol) {
      for (unsigned i_beam_dim1 = 0; i_beam_dim1 != nof_beams_dim1; ++i_beam_dim1) {
        for (unsigned i_beam_dim2 = 0; i_beam_dim2 != nof_beams_dim2; ++i_beam_dim2)
    {
          beam_identifier beam_id = get_beam_id(topology, i_panel, i_pol, i_beam_dim1
    , i_beam_dim2);
          unsigned i_port = nof_elements_dim1 * nof_elements_dim2 * (
    nof_polarizations * i_panel + i_pol);

          for (unsigned j = 0; j != nof_elements_dim1; ++j) {
            cf_t v_l = std::polar(amplitude, TWOPI * j * i_beam_dim1 / nof_beams_dim1
    );
            for (unsigned k = 0; k != nof_elements_dim2; ++k) {
              cf_t u_m   = std::polar(1.0F, TWOPI * k * i_beam_dim2 / nof_beams_dim2)
    ;
              beam_weights.set_coefficient(v_l * u_m, beam_id, i_port++);
            }
          }
        }
      }
    }
  }
  return beam_weights;
}
```

Ausgabe für `single_panel_two_one` (20 Beams × 4 Antennen, gekürzt):

```
beam  0: [+1.00+0.00j  +0.00+0.00j  +0.00+0.00j  +0.00+0.00j]
beam  3: [+0.00+0.00j  +0.00+0.00j  +0.00+0.00j  +1.00+0.00j]
beam  4: [+0.71+0.00j  +0.71+0.00j  +0.00+0.00j  +0.00+0.00j]   l=0, pol 0
beam  5: [+0.71+0.00j  +0.50+0.50j  +0.00+0.00j  +0.00+0.00j]   l=1, pol 0
beam 12: [+0.00+0.00j  +0.00+0.00j  +0.71+0.00j  +0.71+0.00j]   l=0, pol 1
beam 13: [+0.00+0.00j  +0.00+0.00j  +0.71+0.00j  +0.50+0.50j]   l=1, pol 1
```

3.3 Die Zerlegung

Die Steuervektoren $v_{l,m}$ des Type-I-Codebooks sind DFT-Vektoren mit Phase $2\pi li/(N_1 O_1)$. Das Beam-Codebook enthält dieselben Vektoren. Deshalb lässt sich jede 3GPP-Matrix zerlegen:

$$W_{\text{3GPP}}\ (\text{Antennen} \times \text{Layer}) = W_{\text{Beam}}^{\mathsf{T}}\ (\text{Antennen} \times \text{Beams}) \cdot W_{\text{MIMO}}\ (\text{Beams} \times \text{Layer}). \quad (1)$$

W_{MIMO} ist winzig: pro Layer zwei Einträge, einer pro Polarisation mit $\pm\varphi$. Es entsteht in `calculate_mimo_matrix()`:

lib/ran/precoding/type1_sp/precoding_codebooks.cpp, calculate_mimo_matrix()

```
precoding_beam_list beams = get_beams_from_pmi(pmi, nof_layers);   // Beam-IDs aus
    i_1_1, i_1_2, i_1_3
unsigned nof_beams = beams.size() / 2;                              // ohne
    Polarisationsdimension

unsigned i_2 = pmi.i_2 % pmi_ranges.i_2;
```

```
cf_t     phi = std::polar(1.0F, (TWOPI / 4.0F) * static_cast<float>(i_2));

precoding_weight_matrix weights(nof_layers, beams.size());

// Normalize precoding by the number of layers and ports, apply the reciprocal of the
     beamforming scaling.
float scaling = std::sqrt(static_cast<float>(info.n1 * info.n2)) / std::sqrt(
    static_cast<float>(nof_layers * nof_ports));

for (unsigned i_layer = 0; i_layer != nof_layers; ++i_layer) {
  unsigned i_beam    = i_layer % nof_beams;
  // First half of the layers uses +phi, second half uses -phi, as per TS38.214
    Section 5.2.2.2.1.
  cf_t     layer_pol = (i_layer < (nof_layers + 1) / 2) ? phi : -phi;

  weights.set_coefficient(scaling,             i_layer, 2 * i_beam);      //
    Polarisation 0
  weights.set_coefficient(scaling * layer_pol, i_layer, 2 * i_beam + 1);  //
    Polarisation 1
}
return {weights, beams};
```

Der Precoding-Code kennt das Beam-Codebook nicht, nur die Beam-*Nummern* über `get_beam_id()`. Beide Seiten sind ausschließlich über diese Nummerierungskonvention gekoppelt. Dass sie zusammenpassen, sichert der Test `precoding_matrix_factorization_test`.

Zahlenbeispiel `single_panel_two_one`, 2 Layer, $i_{1,1} = 1$, $i_{1,3} = 0$, $i_2 = 1$: beide Layer auf Beam $l = 1$, Beam-IDs 5 (Pol. 0) und 13 (Pol. 1), $\varphi = j$, Skalierung $\sqrt{2}/\sqrt{2 \cdot 4} = 0{,}5$.

$$W_{\text{MIMO}} = \begin{bmatrix} 0{,}5 & +0{,}5j \\ 0{,}5 & -0{,}5j \end{bmatrix} \quad \text{(Layer} \times \text{Beams } 5, 13\text{)}$$

Layer 0 auf den Antennen: $0{,}5 \cdot \text{beam}_5 + 0{,}5j \cdot \text{beam}_{13} = [\,0{,}354,\ 0{,}25 + 0{,}25j,\ 0{,}354j,\ -0{,}25 + 0{,}25j\,]$, identisch mit der Spalte aus `make_type1_sp_mode1()`.

3.4 Wo die Multiplikation mit den IQ-Samples passiert

1. **Upper PHY, Precoding.** Der Resource-Grid-Mapper (`lib/phy/support/resource_grid_mapper_imp` ruft pro PRG `precoder.apply_precoding()`. Mit Beam-Raster ist das W_{MIMO}, ohne die volle 3GPP-Matrix. Die Rechnung steht in `lib/phy/generic_functions/precoding/channel_precoder`

   ```
   for (unsigned i_re = 0; i_re != nof_re; ++i_re) {
     // Set the port RE to the contribution of the first layer.
     cf_t sum = layer_re_view_list[0][i_re] * port_weights[0];
     for (unsigned i_layer = 1; i_layer != nof_layers; ++i_layer) {
       // Accumulate the contributions of all other layers.
       sum += layer_re_view_list[i_layer][i_re] * port_weights[i_layer];
     }
     port_re[i_re] = sum;
   }
   ```

2. **Lower PHY, Beamforming.** Beim OFDM-Modulieren holt `lib/phy/lower/processors/downlink/pd` für jede physische Antenne die Spalte des Beam-Codebooks und der Modulator bildet die gewichtete Summe über die Beam-Ports des Grids. Nullgewichte werden übersprungen (`std::isnormal`-Prüfung in `lib/phy/lower/modulation/ofdm_modulator_impl.cpp:86`).

3.5 Wann `generate_beam_weights_codebook()` aufgerufen wird

Genau einmal pro Zelle beim Start, unabhängig vom Traffic, in der Initialisierungsliste von `pdxch_processor_impl` (`lib/phy/lower/processors/downlink/pdxch/pdxch_processor_impl.h:52`) sowie für Open Fronthaul in `lib/ofh/transmitter/ofh_transmitter_factories.cpp:68`.

lib/phy/lower/processors/downlink/pdxch/pdxch_processor_impl.h

```
pdxch_processor_impl(std::unique_ptr<ofdm_symbol_modulator> modulator,
                     task_executor&                         executor,
                     const configuration&                   config) :
  logger(ocudulog::fetch_basic_logger("PHY")),
  beamforming_codebook(generate_beam_weights_codebook(config.tx_ant_topology)),
  ...
```

Ohne `tx_ant_topology` in der Konfiguration wird die Topologie aus `nof_antennas_dl` abgeleitet: `one_port` bis `eight_ports`. Keine davon hat ein Beam-Raster, das Codebook ist dann die Einheitsmatrix und wirkungslos.

Setup	Beamforming geschieht in	Beam-Codebook
SDR, `four_ports`	Upper PHY, volle 3GPP-Matrix	Einheitsmatrix, wirkungslos
SDR, `single_panel_two_one`	Upper PHY W_{MIMO}, Lower PHY Tabelle	nötig
OFH-RU mit Beamforming	Upper PHY W_{MIMO}, RU Tabelle	nötig, wird an RU gesendet

3.6 Wofür das Beam-Codebook trotzdem existiert

- **Open Fronthaul mit Beamforming in der Radio Unit.** Die DU sendet pro Beam-Port und nennt im C-Plane die Beam-ID. `lib/ofh/support/beamforming_weights_repository.cpp` komprimiert jede Zeile der Tabelle einmal und schickt sie an die RU.
- **Beams ohne PMI.** SSB, PDCCH, SIB und CSI-RS haben kein UE, das einen PMI meldet. Die Zellkonfiguration zeigt mit `ref_beams` direkt auf Zeilen der Tabelle (z. B. SSB-Sweeping).
- **Ein Grid-Format für beide Backends.** Der Upper PHY schreibt immer dasselbe Resource Grid mit Beam-Ports, egal ob dahinter die eigene Lower PHY oder eine OFH-RU hängt.

3.7 Der UHD-SDR-Fall

Mit einem UHD-SDR und Standardtopologie passiert das komplette Beamforming im Upper PHY mit der 3GPP-Matrix:

1. Codebook beim Start: `make_type1_sp_mode1()` für jeden PMI, FAPI-Repository speichert.
2. Precoding pro PDSCH: `apply_precoding_port()`, pro Port eine Spalte von W, pro Resource Element eine Summe über die Layer. Das ist $W \cdot x$ auf Subträgerebene, vor der IFFT.
3. OFDM-Modulation: pro Port IFFT und Cyclic Prefix. Beam-Codebook ist Einheitsmatrix.
4. UHD: die Antennen-Zeitsignale gehen als TX-Kanäle an das Radio. Die relative Phase zwischen den Kanälen ist die Keule; die Hardware muss phasenkohärent senden (gemeinsamer LO, z. B. N310/X310).

Konfiguration: `nof_antennas_dl: 4`, `tx_ant_topology` nicht setzen. Das Codebook ist dann `two_one`, 2 kreuzpolarisierte Elemente, 8 Richtungen über $i_{1,1}$. Für 16 Richtungen mit 4 Elementen pro Polarisation sind 8 TX-Kanäle nötig (`four_one`).

3.8 Was OCUDU beim Beamforming nicht macht

- Keine Keule auf einen beliebigen Winkel in Grad, nur das feste DFT-Raster.
- Keine Nebenkeulenunterdrückung durch Amplituden-Tapering; alle Koeffizienten haben denselben Betrag.
- Kein Downlink-Beamforming aus SRS-Messungen; die SRS-Kanalmatrix dient nur der Uplink-TPMI-Auswahl.
- Keine Nullstellen auf andere UEs, kein Zero-Forcing für Multi-User-MIMO.

Die Richtung bestimmt das UE über den PMI-Report; OCUDU wendet sie an. Im Testmodus setzt man sie mit $i_{1,1}$ fest.

4 Die Mathematik von `make_type1_sp_mode1()`

TS 38.214 Abschnitt 5.2.2.2.1, Type I Single Panel, Codebook-Mode 1.

4.1 Antennenmodell

Ein Panel mit N_1 Elementen horizontal, N_2 vertikal, jedes kreuzpolarisiert. CSI-RS-Ports $P = 2N_1N_2$. Die ersten N_1N_2 Ports sind Polarisation 0, die zweiten Polarisation 1. O_1, O_2 sind die Oversampling-Faktoren; die Zahl der Richtungen ist N_1O_1 horizontal und N_2O_2 vertikal.

Codebook	N_1	N_2	O_1	O_2	Ports	Richtungen
`two_one`	2	1	4	1	4	8
`two_two`	2	2	4	4	8	8×8
`four_one`	4	1	4	1	8	16

4.2 Der Steuervektor $v_{l,m}$

$$u_m = \begin{bmatrix} 1 & e^{j\frac{2\pi m}{O_2 N_2}} & \cdots & e^{j\frac{2\pi m(N_2-1)}{O_2 N_2}} \end{bmatrix} \tag{2}$$

$$v_{l,m} = \begin{bmatrix} u_m & u_m\, e^{j\frac{2\pi l}{O_1 N_1}} & \cdots & u_m\, e^{j\frac{2\pi l(N_1-1)}{O_1 N_1}} \end{bmatrix}^{\mathsf{T}} \tag{3}$$

Ein Kronecker-Produkt aus horizontalem und vertikalem DFT-Vektor, Länge N_1N_2. l wählt die horizontale Richtung, m die vertikale. Alle Elemente haben Betrag 1, es gibt kein Tapering. Bei Elementabstand $d = \lambda/2$ liegt die Hauptkeule bei $\sin\theta = 2l/(O_1N_1)$ für $l \le N_1O_1/2$, negative Winkel für die obere Hälfte von l.

Im Code wird der Vektor durch fortlaufende Multiplikation mit einem Phasor aufgebaut:

lib/ran/precoding/type1_sp/precoding_codebooks.cpp, make_layer_type1_sp_mode1()

```
static void make_layer_type1_sp_mode1(precoding_weight_matrix&             result,
                                      const pmi_codebook_single_panel_info& panel,
                                      unsigned                             i_layer,
                                      unsigned                             l,
                                      unsigned                             m,
```

```
                                        cf_t
    polarization_offset)
{
  // Phase increment for each beam coefficient. This defines the direction of the
    beam in the horizontal plane.
  float phase_inc_h_rad = TWOPI * (static_cast<float>(l) / static_cast<float>(panel.
    o1 * panel.n1));
  // Phase increment for each beam coefficient. This defines the direction of the
    beam in the vertical plane.
  float phase_inc_v_rad = TWOPI * (static_cast<float>(m) / static_cast<float>(panel.
    o2 * panel.n2));
  // Get the number of physical ports, which is half the number of CSI-RS ports given
     that two polarizations are used.
  unsigned nof_ports_2 = result.get_nof_ports() / 2;

  cf_t phase_inc_h = std::polar(1.0F, phase_inc_h_rad);
  cf_t phase_inc_v = std::polar(1.0F, phase_inc_v_rad);
  cf_t beam_coef_h = std::polar(1.0F, 0.0F);
  cf_t beam_coef_v = std::polar(1.0F, 0.0F);

  unsigned i_port = 0;
  for (unsigned i_h = 0; i_h != panel.n1; ++i_h) {
    beam_coef_v = std::polar(1.0F, 0.0F);
    for (unsigned i_v = 0; i_v != panel.n2; ++i_v) {
      cf_t beam_coef = beam_coef_h * beam_coef_v;
      // First polarization.
      result.set_coefficient(beam_coef, i_layer, i_port);
      // Second polarization, same beam but with phase offset.
      result.set_coefficient(polarization_offset * beam_coef, i_layer, nof_ports_2 +
    i_port);
      i_port++;
      beam_coef_v *= phase_inc_v;
    }
    beam_coef_h *= phase_inc_h;
  }
}
```

4.3 Die Co-Phasierung φ_n

$$\varphi_n = e^{j\pi n/2}, \qquad n = i_2 \in \{0,1,2,3\} \quad \Rightarrow \quad \varphi \in \{1,\, j,\, -1,\, -j\} \tag{4}$$

Die Phase der zweiten Polarisation relativ zur ersten. Im Code `phi = std::polar(1, (TWOPI/4)*i_2)`.

4.4 Ein Layer

$$W^{(1)} = \frac{1}{\sqrt{P}} \begin{bmatrix} v_{l,m} \\ \varphi_n v_{l,m} \end{bmatrix} \qquad (P \times 1), \quad l = i_{1,1},\ m = i_{1,2} \tag{5}$$

Tabelle 5.2.2.2.1-5.

4.5 Zwei Layer

$$W^{(2)} = \frac{1}{\sqrt{2P}} \begin{bmatrix} v_{l,m} & v_{l',m'} \\ \varphi_n v_{l,m} & -\varphi_n v_{l',m'} \end{bmatrix} \qquad (P \times 2), \quad l' = l + k_1,\ m' = m + k_2 \tag{6}$$

Die Offsets (k_1, k_2) kommen aus $i_{1,3}$ über Tabelle 5.2.2.2.1-3, im Code `get_k1_and_k2_2_layers()`. Für `two_one` ist $k_1 \in \{0, O_1\}$: derselbe Beam oder der nächste Hauptkeulen-Nachbar. Das Minus

in der zweiten Spalte macht die Layer orthogonal, auch bei $k_1 = 0$: $[v; \varphi v] \perp [v; -\varphi v]$. i_2 ist hier nur noch 0 oder 1.

4.6 Drei und vier Layer

($P < 16$)

$$W^{(3)} = \frac{1}{\sqrt{3P}} \begin{bmatrix} v_{l,m} & v_{l',m'} & v_{l,m} \\ \varphi_n v_{l,m} & \varphi_n v_{l',m'} & -\varphi_n v_{l,m} \end{bmatrix} \tag{7}$$

$$W^{(4)} = \frac{1}{\sqrt{4P}} \begin{bmatrix} v_{l,m} & v_{l',m'} & v_{l,m} & v_{l',m'} \\ \varphi_n v_{l,m} & \varphi_n v_{l',m'} & -\varphi_n v_{l,m} & -\varphi_n v_{l',m'} \end{bmatrix} \tag{8}$$

(k_1, k_2) nach Tabelle 5.2.2.2.1-4, `get_k1_and_k2_3_4_layers()`; hier ist $k = 0$ nicht erlaubt.

make_type1_sp_mode1(), Auswahl der Spalten

```
switch (nof_layers) {
  case 1: {
    layer_beam_horizontal = {i_1_1};
    layer_beam_vertical   = {i_1_2};
    layer_pol             = {phi};
    break;
  }
  case 2: {
    std::tie(k1, k2) = get_k1_and_k2_2_layers(panel_info, i_1_3);
    layer_beam_horizontal = {i_1_1, i_1_1 + k1};
    layer_beam_vertical   = {i_1_2, i_1_2 + k2};
    layer_pol             = {phi, -phi};
    break;
  }
  case 3:
  case 4: {
    std::tie(k1, k2)      = get_k1_and_k2_3_4_layers(panel_info, i_1_3);
    layer_beam_horizontal = {i_1_1, i_1_1 + k1, i_1_1, i_1_1 + k1};
    layer_beam_vertical   = {i_1_2, i_1_2 + k2, i_1_2, i_1_2 + k2};
    layer_pol             = {phi, phi, -phi, -phi};
    break;
  }
  // ... Faelle 5 bis 8: Tabellen 5.2.2.2.1-9 bis -12 ...
}

// Build the coefficients for each port and layer.
for (unsigned i_layer = 0; i_layer != nof_layers; ++i_layer) {
  make_layer_type1_sp_mode1(result, panel_info, i_layer,
                            layer_beam_horizontal[i_layer],
                            is_2d_panel ? layer_beam_vertical[i_layer] : 0,
                            layer_pol[i_layer]);
}

// Apply scaling.
float scaling = 1.0F / std::sqrt(nof_layers * nof_ports);
result *= scaling;
```

Fünf bis acht Layer benutzen bis zu vier Beams mit festen Offsets O_1 und $2O_1$ und abwechselnd φ und ± 1 (Tabellen 5.2.2.2.1-9 bis -12).

4.7 Normierung

$$\text{scaling} = \frac{1}{\sqrt{\upsilon P}} \tag{9}$$

υ Layer, P Ports. Jeder Eintrag hat Betrag $1/\sqrt{\upsilon P}$, die Sendeleistung über alle Ports und Layer ist 1.

4.8 Zahlenbeispiel

`two_one`, 2 Layer, $i_{1,1} = 1$, $i_{1,3} = 0$, $i_2 = 1$:

$$v_1 = \begin{bmatrix} 1 \\ e^{j2\pi/8} \end{bmatrix} = \begin{bmatrix} 1 \\ 0{,}707 + 0{,}707j \end{bmatrix}, \quad \varphi = j, \quad k_1 = 0$$

$$W^{(2)} = \frac{1}{\sqrt{8}} \begin{bmatrix} 1 & 1 \\ 0{,}707 + 0{,}707j & 0{,}707 + 0{,}707j \\ j & -j \\ -0{,}707 + 0{,}707j & 0{,}707 - 0{,}707j \end{bmatrix}$$

Spalte 1 ist $[\,0{,}354,\ 0{,}25 + 0{,}25j,\ 0{,}354j,\ -0{,}25 + 0{,}25j\,]$, derselbe Vektor wie aus der Zerlegung in Abschnitt 3.

4.9 Was das Precoding damit macht

$$y_p[k] = \sum_{\upsilon} W[p, \upsilon]\, x_\upsilon[k] \qquad \text{für jeden Subträger } k \text{ und jeden Port } p \tag{10}$$

Das ist die Schleife in `apply_precoding_port()`: pro Port eine Zeile von W, pro Resource Element die Summe über die Layer. Am Empfänger in Richtung θ ergibt sich der Array-Faktor

$$AF(\theta) = \sum_{i=0}^{N_1 - 1} e^{j2\pi i \left(\frac{l}{O_1 N_1} - \frac{d \sin\theta}{\lambda} \right)}, \tag{11}$$

maximal, wenn die Klammer null ist. Das ist die Hauptkeule; Nullstellen und Nebenkeulen folgen aus der Summe über N_1 Elemente.

5 Die beiden Dateien `precoding_codebooks.cpp`

Zwei Dateien gleichen Namens mit klar getrennten Rollen: oben die Fassade und die codebook-unabhängigen Matrizen, unten die eigentliche 3GPP-Type-I-Mathematik.

5.1 `lib/ran/precoding/precoding_codebooks.cpp`, die Fassade

Implementiert den öffentlichen Header `include/ocudu/ran/precoding/precoding_codebooks.h`.

- Einfache Matrizen ohne PMI: `make_single_port()`, `make_one_layer_one_port()`, `make_one_layer_all` `make_identity()`. Sie braucht der FAPI-Adaptor für SSB, PDCCH, CSI-RS und das omnidirektionale PDSCH.
- `get_single_panel_topology()`: Abbildung von 2×1, 2×2, 4×1 auf die Antennentopologie.
- Zwei Dispatcher: `make_precoding()` und `get_mimo_matrix_from_pmi()`. Die PMI ist ein `std::variant`; die Visitor-Structs rufen die passende Implementierung.

lib/ran/precoding/precoding_codebooks.cpp

```
/// Precoding weight matrix calculator from a given PMI.
struct precoding_calculator {
  unsigned nof_layers;

  precoding_weight_matrix operator()(const pmi_two_antenna_port& pmi) const
  {
    switch (nof_layers) {
      case 1: return make_one_layer_two_ports(pmi.pmi);
      case 2: return make_two_layer_two_ports(pmi.pmi);
      default: ocudu_assertion_failure("Unsupported PMI codebook configuration");
    }
    return {};
  }

  precoding_weight_matrix operator()(const pmi_typeI_single_panel& pmi) const
  {
    return make_type1_sp_mode1(pmi, nof_layers);
  }

  precoding_weight_matrix operator()(const pmi_typeII& pmi) const { return make_type2
    (pmi, nof_layers); }
};

precoding_weight_matrix ocudu::make_precoding(const precoding_matrix_indicator& pmi,
    unsigned nof_layers)
{
  return std::visit(precoding_calculator{nof_layers}, pmi);
}
```

Hier wird nichts gerechnet, nur verteilt.

5.2 `lib/ran/precoding/type1_sp/precoding_codebooks.cpp`, die Umsetzung

Die Implementierung von TS 38.214 Abschnitt 5.2.2.2.1 für ein Single Panel:

- `make_layer_type1_sp_mode1()`: ein Steuervektor $[v_{l,m};\ \varphi v_{l,m}]$ für einen Layer.
- `get_k1_and_k2_2_layers()`, `get_k1_and_k2_3_4_layers()`: Tabellen 5.2.2.2.1-3 und -4.
- `make_type1_sp_mode1()`: die vollständige Matrix Ports × Layer für 1 bis 8 Layer.
- `get_beams_from_pmi()`, `calculate_mimo_matrix()`: die zerlegte Variante für Beam-Raster.

Der zugehörige Header `type1_sp/precoding_codebooks.h` liegt bewusst in `lib/` statt in `include/`: Diese Funktionen sind intern, von außen soll nur über `make_precoding()` zugegriffen werden.

5.3 Warum die Aufteilung

Der Rest des Systems soll nicht wissen, welcher Codebook-Typ gilt. Er ruft `make_precoding(pmi, nof_layers)` und bekommt eine Matrix. Jeder Codebook-Typ hat sein eigenes Unterverzeichnis mit eigener Datei desselben Namens. Zum Lesen geht man von `precoding_codebooks.cpp:151` in den Visitor und von dort in `type1_sp/precoding_codebooks.cpp`. Die dritte Datei, `type1_sp/precoding_co` liefert die Wertebereiche und Bitbreiten der PMI-Felder nach TS 38.212 Tabelle 6.3.1.1.2-1, die der FAPI-Adaptor für seine Schleifen und der CSI-Report-Decoder brauchen.

6 Die Tests: Parametrisierung und Matrix-Ausgabe

6.1 Wer über die Topologien iteriert

`tests/unittests/ran/beamforming/beam_codebook_generator_test.cpp` enthält keine Schleife über die Topologien; der Test wird sieben Mal als eigenständiger Testfall ausgeführt. Der Iterator ist Google Test.

tests/unittests/ran/beamforming/beam_codebook_generator_test.cpp

```
class BeamCodebookGeneratorFixture : public ::testing::TestWithParam<antenna_topology
    >
{};

TEST_P(BeamCodebookGeneratorFixture, Generate)
{
  antenna_topology topology = GetParam();
  // ...
}

static constexpr auto antenna_topologies = to_array<antenna_topology>({
    antenna_topology::one_port,

    antenna_topology::two_port,

    antenna_topology::four_ports,

    antenna_topology::eight_ports,

    antenna_topology::single_panel_two_one,

    antenna_topology::single_panel_two_two,

    antenna_topology::single_panel_four_one});

INSTANTIATE_TEST_SUITE_P(BeamCodebookGenerator, BeamCodebookGeneratorFixture, ::
    testing::ValuesIn(antenna_topologies));
```

1. Das Array ist nur die Datenquelle.
2. `INSTANTIATE_TEST_SUITE_P` ist der Auslöser. `ValuesIn()` verpackt das Array in einen `ParamGenerator<antenna_topology>` (intern `ValuesInIteratorRangeGenerator`). Das Makro legt ein statisches Objekt an, das den Generator bei Programmstart in der gtest-Registry hinterlegt.
3. Die Schleife steckt in gtest, `/usr/include/gtest/internal/gtest-param-util.h`, `ParameterizedTestS` ausgeführt bei `InitGoogleTest()`:

   ```
   for (typename ParamGenerator<ParamType>::iterator param_it = generator.begin();
        param_it != generator.end(); ++param_it, ++i) {
     // ...
     test_name_stream << test_info->test_base_name << "/" << param_name;
     MakeAndRegisterTestInfo(test_suite_name.c_str(), test_name_stream.GetString()
       .c_str(), ...);
   }
   ```

   Pro Wert entsteht ein Testfall `BeamCodebookGenerator/BeamCodebookGeneratorFixture.Generate/0` bis `/6`, jeder mit einer Factory, die den Parameterwert gespeichert hat.

4. Zur Laufzeit erzeugt die Factory pro Testfall ein frisches Fixture-Objekt und setzt den Parameter. `GetParam()` liest ihn nur aus.

```
./cmake-build-debug/tests/unittests/ran/beam_codebook_generator_test --
    gtest_list_tests
./cmake-build-debug/tests/unittests/ran/beam_codebook_generator_test --gtest_filter='
    */4'   # single_panel_two_one
```

6.2 Beam-Gewichte als Matrix ausgeben

Für `cf_t` gibt es in `ocudu/adt/format.h` einen Formatter; `{:+.2f}` ergibt z. B. `+0.50-0.50j`. Für `beam_weights_codebook` existiert kein Formatter, anders als für `precoding_weight_matrix` (`include/ocudu/ran/precoding/precoding_weight_matrix_formatters.h`). Direkt im Test, eine Zeile pro Beam:

```
fmt::print("Topologie {}: {} Beams x {} Antennen\n", to_string(topology),
    beam_weights.get_nof_beams(), nof_total_antennas);
for (unsigned i_beam = 0; i_beam != beam_weights.get_nof_beams(); ++i_beam) {
  fmt::print("  beam {:2d}: [", i_beam);
  for (unsigned i_antenna = 0; i_antenna != nof_total_antennas; ++i_antenna) {
    fmt::print("{}{:+.2f}", (i_antenna != 0) ? "  " : "",
               beam_weights.get_coefficient(static_cast<beam_identifier>(i_beam),
    i_antenna));
  }
  fmt::print("]\n");
}
```

Die erwartete Zeile im selben Layout, an der Stelle nach der Berechnung von `expected_coefficients`:

```
fmt::print("  expected beam {:2d} (panel={} pol={} l={} m={}): [",
           to_underlying(beam_id), i_panel, i_pol, i_beam_dim1, i_beam_dim2);
for (unsigned i_antenna = 0; i_antenna != nof_total_antennas; ++i_antenna) {
  fmt::print("{}{:+.2f}", (i_antenna != 0) ? "  " : "", expected_coefficients[
    i_antenna]);
}
fmt::print("]\n");
```

Die ersten Zeilen bis zur Antennenzahl sind die Einheitsmatrix (direkte Port-Beams), danach die DFT-Beams nach Panel, Polarisation, l und m. Die Zeilennummer ist die `beam_identifier`.